Manipulación de API Rest

Consumo de APIs REST en React Native

1. Introducción

En esta clase vas a aprender:

- Qué son los hooks intermedios y por qué son útiles.
- Cómo usar useRef, useMemo y useCallback.
- Qué es un Custom Hook y cómo crear lógica reutilizable.
- Principios de composición lógica y separación de preocupaciones.
- Casos de uso prácticos en apps React Native con Expo.

2. ¿Qué son los Hooks Intermedios?

Los hooks intermedios permiten controlar con más precisión la lógica de renderizado, el rendimiento y la persistencia de datos dentro de componentes.

Se utilizan cuando:

- Hay funciones que se ejecutan múltiples veces sin necesidad.
- Se necesita almacenar valores sin volver a renderizar.
- Se desea optimizar funciones o cálculos costosos.
- Se quiere reutilizar lógica en múltiples componentes.

3. useRef

3.1 ¿Qué es?

useRef permite mantener una referencia mutable que no causa re-render al cambiar su valor. Es útil para:

- Acceder a elementos visuales (TextInput, View, etc.).
- Almacenar valores entre renders (como temporizadores o contadores).

3.2 Sintaxis y ejemplo

```
import { useRef } from 'react';
import { TextInput, Button } from 'react-native';
```

```
export default function Formulario() {
  const inputRef = useRef(null);

  const enfocarInput = () => {
    inputRef.current?.focus();
  };

  return (
    <>
      <TextInput ref={inputRef} placeholder="Tu nombre" />
      <Button title="Enfocar" onPress={enfocarInput} />
    </>
  );
}
```

3.3 Características

- No genera re-render al cambiar su valor.
- Su valor persiste entre renders.
- Muy útil en animaciones, intervalos y referencias DOM.

4. useMemo

4.1 ¿Qué es?

useMemo memoriza el resultado de una función costosa para evitar volver a calcularla si sus dependencias no han cambiado.
Ideal para cálculos pesados o filtros que dependen de valores que rara vez cambian.

4.2 Sintaxis y ejemplo

```
const usuariosFiltrados = useMemo(() => {
  return listaUsuarios.filter(u => u.edad > 18);
}, [listaUsuarios]);
```

4.3 Características

Ventaja	Explicación
Mejora de rendimiento	Evita recálculo innecesario en renders.
Control por dependencias	Solo recalcula si algo cambia.
Útil con listas grandes	Especialmente con FlatList y filtros.

5. useCallback

5.1 ¿Qué es?

useCallback memoriza funciones. Es útil cuando se pasa una función como prop a un componente hijo y se quiere evitar su recreación constante.
Ideal junto a React.memo() para prevenir renders innecesarios.

5.2 Sintaxis y ejemplo

```
const incrementar = useCallback(() => {
  setContador(prev => prev + 1);
}, []);
```

5.3 Diferencias con useMemo

Hook	Devuelve...	Uso principal
useMemo	Resultado	Memorizar cálculos
useCallback	Función	Memorizar funciones

6. Custom Hooks

6.1 ¿Qué es un Custom Hook?

Un Custom Hook es una función que empieza con use y permite reutilizar lógica con estado o efectos en múltiples componentes.
Sirve para separar la lógica del componente de su presentación visual.

6.2 Ejemplo: useContador

```
import { useState } from 'react';

export function useContador(inicial = 0) {
  const [valor, setValor] = useState(inicial);

  const incrementar = () => setValor(prev => prev + 1);
  const decrementar = () => setValor(prev => prev - 1);
  const resetear = () => setValor(inicial);

  return { valor, incrementar, decrementar, resetear };
}
```

Uso:

```
const { valor, incrementar } = useContador(5);
```

6.3 Beneficios de usar custom hooks

Beneficio	Descripción
Reutilización de lógica	Se evita repetir useState, useEffect, etc.
Código más limpio	Los componentes quedan más legibles.
Separación de responsabilidades	Cada hook se enfoca en una sola tarea.
Testeo más fácil	Se puede testear la lógica sin renderizar UI.

6.4 Ejemplo práctico: useCargarDatos

```
import { useEffect, useState } from 'react';

export function useCargarDatos(url) {
  const [datos, setDatos] = useState([]);
  const [cargando, setCargando] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch(url)
      .then(resp => resp.json())
      .then(setDatos)
      .catch(setError)
      .finally(() => setCargando(false));
  }, [url]);

  return { datos, cargando, error };
}
```

Uso:

```
const { datos, cargando, error } =
useCargarDatos('https://jsonplaceholder.typicode.com/users');
```

7. Composición lógica

7.1 ¿Qué es la composición lógica?

Es el principio de desacoplar la lógica de negocio de los componentes visuales, organizando la aplicación en funciones reutilizables (hooks), en lugar de copiar y pegar lógica entre pantallas.
En vez de crear componentes gigantes, usamos hooks para dividir lógica en módulos.

7.2 Ejemplo de composición

```
// En hooks/useAuth.js
```

```
export function useAuth() {
  const [usuario, setUsuario] = useState(null);

  const login = (credenciales) => { /* login logic */ };
  const logout = () => setUsuario(null);

  return { usuario, login, logout };
}
```

```
// En LoginScreen.jsx
const { login } = useAuth();
```

8. Buenas prácticas



BUENAS PRÁCTICAS

Todos los **custom hooks deben empezar con use.**

Evitá usar hooks dentro de condiciones o bucles.

Reutilizá lógica repetida creando hooks propios.

Documentá cada hook: entrada, salida y efectos secundarios.

Usá useCallback o useMemo cuando trabajes con listas o componentes hijos que se renderizan frecuentemente.



Bibliografía utilizada y sugerida

Meta Platforms. React – Hooks API Reference.
https://react.dev/reference/react
Actualizado en Julio 2024.

Meta Platforms. React Native – Optimizing Performance.
https://reactnative.dev/docs/getting-started
Actualizado en Julio 2024.

Kent C. Dodds. Guide to useMemo and useCallback.
https://kentcdodds.com/blog/usememo-and-usecallback
Actualizado en 2024.